ETH zürich



Exercise Session 11 – Dynamic Programming

Computer Science II

April 29 / 30, 2025

Program Today

- Mars Mission
- King's way
- Greedy king's way

Last week: dynamic programming

Do you still have questions?

- Tribonacci
- Catalan numbers
- Blocks
- Task scheduling v2.0

1. Mars Mission

Problem description

The task involves a rover named Perseverance which has landed on Mars at a position labeled as S (start).

S	9	2	5	11	8
17	21	32	5	15	3
2	2	3	8	1	5
8	2	8	11	15	9
0	5	3	10	4	Z

Problem description

The rover aims to reach a target position Z (end)

S	9	2	5	11	8
17	21	32	5	15	3
2	2	3	8	1	5
8	2	8	11	15	9
0	5	3	10	4	Z

Problem description

The rover can only move to the east (right) or south (down).

S	9	2	5	11	8
17	21	32	5	15	3
2	2	3	8	1	5
8	2	8	11	15	9
0	5	3	10	4	Z

Problem description

Each cell on the grid contrains a value representing a rock sample, which can be collected by the rover if it passes through that cell.

S	9	2	5	11	8
17	21	32	5	15	3
2	2	3	8	1	5
8	2	8	11	15	9
0	5	3	10	4	Z

Problem description

The objective is to find a path from S to Z that maximizes the total value of collected rock samples

S	9	2	5	11	8
17	21	32	5	15	3
2	2	3	8	1	5
8	2	8	11	15	9
0	5	3	10	4	Z

Problem description

- **Input:** A $m \times n$ matrix A.

Problem description

- **Input:** A $m \times n$ matrix A.
- **Output 1:** Maximum possible value that can be collected from stones lying in a south-east path from $(0, 0)$ to $(m - 1, n - 1)$.

Problem description

- **Input:** A $m \times n$ matrix A.
- **Output 1:** Maximum possible value that can be collected from stones lying in a south-east path from $(0, 0)$ to $(m - 1, n - 1)$.
- **Output 2:** The path corresponding to that maximum value.

Step 1: identify subproblems

What are the subproblems?

Step 2: identify recurrence

What are your options when you are at position (i, j)?

Step 2: identify recurrence

Use the options to define a recurrence.

Step 3: implementation

Data structure. What data structure do you need to store the solutions to the subproblems?

Step 3: implementation

Dependencies. To compute $S(i, j)$, what do you need?

Step 3: implementation

Calculation order. How should you fill S?

Step 3: implementation

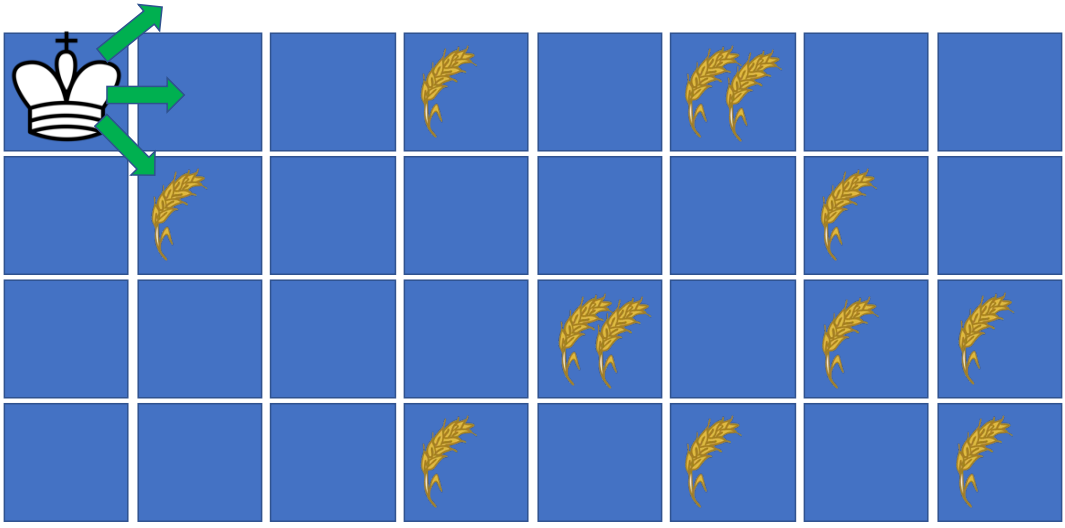
Extracting the solution.

Step 3: implementation

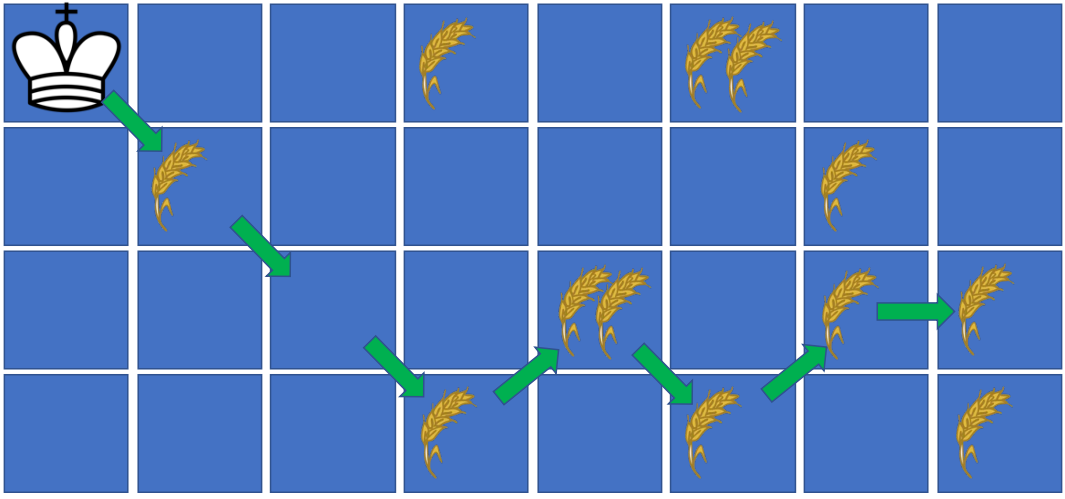
What is the running time?

2. King’s way

Problem description



Problem description



Problem description

- A king is located at cell $(0, 0)$ of a $m \times n$ chessboard.

Problem description

- A king is located at cell $(0, 0)$ of a $m \times n$ chessboard.
- The king can only move northeast, east or southeast. That is, if the king is at position (i, j), then the king can only move to $(i - 1, j + 1)$, $(i, j + 1)$, or $(i + 1, j + 1)$, as long as he does not leave the board.

Problem description

- A king is located at cell $(0, 0)$ of a $m \times n$ chessboard.
- The king can only move northeast, east or southeast. That is, if the king is at position (i, j), then the king can only move to $(i - 1, j + 1)$, $(i, j + 1)$, or $(i + 1, j + 1)$, as long as he does not leave the board.
- Each cell is marked with a reward.

Problem description

- A king is located at cell $(0, 0)$ of a $m \times n$ chessboard.
- The king can only move northeast, east or southeast. That is, if the king is at position (i, j), then the king can only move to $(i - 1, j + 1)$, $(i, j + 1)$, or $(i + 1, j + 1)$, as long as he does not leave the board.
- Each cell is marked with a reward.
- What is the best path that the king can follow to attain the maximum reward?

Problem description

- A king is located at cell $(0, 0)$ of a $m \times n$ chessboard.
- The king can only move northeast, east or southeast. That is, if the king is at position (i, j), then the king can only move to $(i - 1, j + 1)$, $(i, j + 1)$, or $(i + 1, j + 1)$, as long as he does not leave the board.
- Each cell is marked with a reward.
- What is the best path that the king can follow to attain the maximum reward?
- Rewards are non-negative.

Problem description

- **Input:** a 2D-list `a` of size $m \times n$ of non-negative integers.

Problem description

- **Input:** a 2D-list `a` of size $m \times n$ of non-negative integers.
- **Output:** Maximum value that the king can attain and the sequence of moves to attain it.

Step 1: identify subproblems

What are the subproblems?

Step 2: identify recurrence

What are my options?

Step 2: identify recurrence

Use the options to define a recurrence.

Step 3: implementation

Data structure. What data structure do you need to store the solutions to the subproblems?

Step 3: implementation

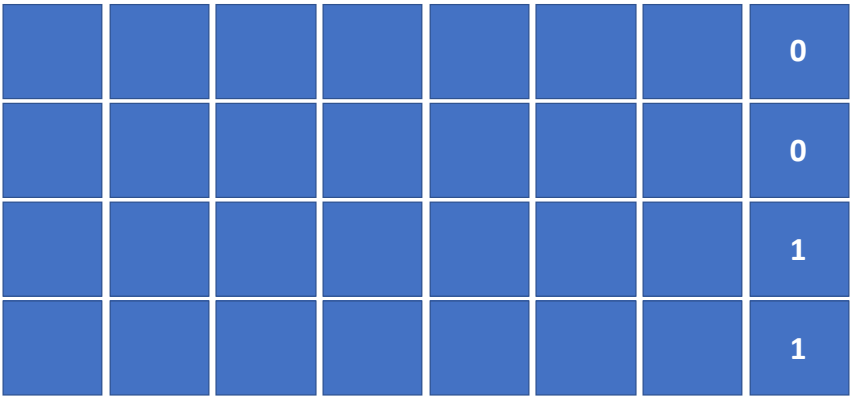
Dependencies. To compute $S(i, j)$, what do you need?

Step 3: implementation

Calculation order. How should you fill `s`?

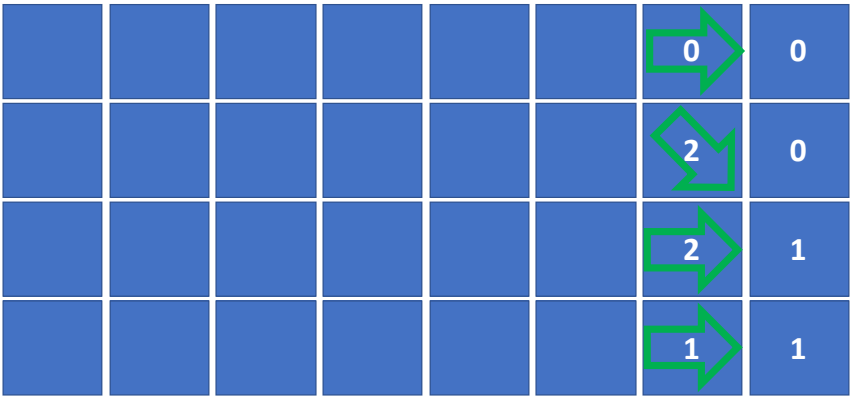
Step 3: implementation

Fill the matrix **s** for the matrix given at the beginning of the section. Keep track of the optimal decision for each subproblem.



Step 3: implementation

Fill the matrix s for the matrix given at the beginning of the section. Keep track of the optimal decision for each subproblem.



Step 3: implementation

Python implementation in code expert.

3. Greedy king’s way

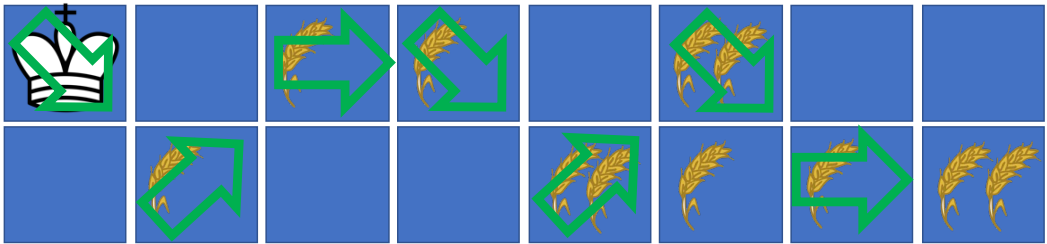
Problem description

What if you only have 2 rows?



Problem description

Do you really need DP?



Greedy solution

When the king is at position (i, j)...

Greedy solution

Strategy: Use a for loop to move the king to the next cell with the highest value.



What's the invariant for this greedy solution?

Implementation

Python implementation in code expert.

4. Homework

Exercise 9: Dynamic programming

On https://expert.ethz.ch/enrolled/SS25/mavt2/exercises

- Mission Mars with lava
- Binomial coefficients
- Triangle
- Longest common substring

Due date: Monday 28.04.2025, 20:00 CET **NO HARDCODING**